AZIENDA TUTELA DELLA SALUTE LIGURIA

SPECIFICHE TECNICHE

Sistema SeiPDF Management

Gestione automatizzata del ciclo documentale

(PDF → ZIP → Invio FTP → Gestione esiti)

Tabella revisioni documento

Versione	Revisione	Data	Autore	Descrizione modifica
1.0	Alessio Chiappini	12/02/2026	Francesco Matraxia	Prima stesura
1.1	Alessio Chiappini	09/11/2026	Francesco Matraxia	Implementazione pipeline CI/CD

Sommario

Tabella revisioni documento 1
1. Introduzione 8
1.1 Scopo del documento 8
1.2 Campo di applicazione 8
1.3 Definizioni e acronimi 8
1.4 Riferimenti normativi o documentali 9
Specifica_Test_SeiPDF.pdf 9
2. Panoramica del sistema 10
2.1 Descrizione generale 10
2.2 Contesto applicativo 10
2.3 Sistemi esterni coinvolti 10
2.4 Stakeholder e attori 11
Attori tecnici 11
Attori applicativi 11
Utenti operativi (indiretti) 11
3. Architettura del sistema 12
3.1 Architettura logica 12
Descrizione dell'architettura a livelli 13
3.2 Architettura fisica 14
Struttura filesystem 14
Hosting applicativo 15
3.3 Componenti applicativi 15
SeiPDF Management (.NET 8) 15
Scheduler (Windows Task Scheduler) 16
Servizio FTP (Java + Tomcat) 16
Database Oracle 16
Filesystem 17
3.4 Tecnologie utilizzate 17
4. Componenti software 18

4.1 API SeiPDF (ASP.NET Core) 18
Responsabilità principali 18
Endpoint principali 18
Caratteristiche tecniche 18
Sicurezza 18
4.2 Scheduler PowerShell 18
Responsabilità principali 19
Script principali 19
Caratteristiche tecniche 19
4.3 Servizio FTP Java su Tomcat 19
Responsabilità principali 19
Modalità di esecuzione 19
Caratteristiche tecniche 20
4.4 Database Oracle 20
Responsabilità principali 20
Tipologie di dati gestiti 20
Interazione con il sistema 20
4.5 Filesystem di integrazione 20
Responsabilità principali 20
Struttura delle directory 21
Directory operative 21
DaInviare 21
Inviati 21
Flusso dei file 21
Caratteristiche operative 22
5. Flussi applicativi 23
5.1 Generazione PDF 23
Flusso operativo 23
Regole di naming 23
Gestione stati database 23
Logging 24
5.2 Creazione pacchetti ZIP 24

Flusso operativo ... 24
Regole di creazione ZIP ... 24
Vincoli applicati ... 24
Operazioni per ogni ZIP ... 25
Gestione stati ... 25
5.3 Invio file tramite FTP ... 25
Flusso operativo ... 25
Gestione file `.t` ... 26
Gestione errori ... 26
Logging e tracciamento ... 26
Output ... 27
5.4 Gestione esiti ... 27
Attivazione ... 27
Flusso operativo ... 27
Elaborazione email ... 27
Aggiornamento sistema ... 27
Caratteristiche tecniche ... 28
5.5 Diagramma di sequenza ... 28
5.6 Diagramma a stati ... 30
Descrizione degli stati ... 30
5.7 Diagramma di flusso ... 32
Descrizione ... 33
6. API REST ... 34
6.1 Elenco endpoint ... 34
6.2 Parametri di input ... 34
Body ... 34
Header opzionale ... 34
Configurazione runtime ... 34
6.3 Struttura delle risposte ... 35
Health ... 35
Export PDF ... 35
Create ZIP ... 35

Mail (download-unread) 36
6.4 Gestione errori 36
Codici HTTP 36
Formato errore 36
Gestione resilienza 37
6.5 Endpoint applicativi non esposti tramite Swagger 37
Invio FTP 37
Descrizione 37
Modalità di utilizzo 37
Regole operative 38
Output 38
Logging 38
7. Scheduler e processi batch 39
7.1 Descrizione job schedulati 39
Elenco job 39
Sequenza logica 39
7.2 Script PowerShell 39
Caratteristiche principali 39
Script principali 40
Invio FTP 40
7.3 Frequenza esecuzione 40
Considerazioni operative 41
7.4 Logging scheduler 41
Tipologie di log 41
Struttura log 41
Obiettivi del logging 42
8. Configurazione sistema 43
8.1 Configurazione IIS 43
Modalità di hosting 43
Configurazione sito 43
Application Pool 43
Configurazione applicative e variabili d'ambiente 44

Autenticazione 44
Logging 44
Aggiornamento applicativo 45
8.2 Configurazione Tomcat 45
Deploy applicazione 45
Configurazione FTP 45
Parametri principali 46
Directory locali 46
Strumenti di invio 46
Logging 46
8.3 Configurazione Oracle 46
Connessione 46
Utilizzo 47
Accessi richiesti 47
8.4 Parametri applicativi 47
Sicurezza 47
Export PDF 47
Creazione ZIP 48
Mail 48
Logging 48
9. Sicurezza 49
9.1 Gestione accessi 49
9.2 Permessi filesystem 49
9.3 Protezione API 49
10. Performance e limiti 51
10.1 Limiti dimensionali ZIP 51
10.2 Numero massimo documenti 51
10.3 Timeout processi 51
Caratteristiche di resilienza 51
11. Deployment 52
11.1 Installazione API su IIS 52
11.2 Installazione servizio Java su Tomcat 55

11.3 Configurazione scheduler 55
11.4 Configurazione connessione database 56
12. Requisiti di sistema 57
Requisiti hardware 57
Requisiti software 57
13. Ambiente di esercizio 58
13.1 Componenti dell'ambiente 58
Server applicativo 58
Server database 58
Server FTP Postel 58
Sistema di posta elettronica 59
Filesystem applicativo 59
13.2 Modalità di esecuzione 59
Schedulazione 59
13.3 Considerazioni operative 59
13.4 Sintesi componenti 60

1. Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere in maniera dettagliata l'architettura, il funzionamento e le modalità di configurazione del sistema **SeiPDF Management**.

In particolare, il documento fornisce le informazioni tecniche necessarie per:

- comprendere il funzionamento complessivo del sistema;
- installare e configurare i componenti applicativi;
- gestire i processi batch e le integrazioni con sistemi esterni;
- supportare le attività di manutenzione e troubleshooting.

Il documento è rivolto a personale tecnico, amministratori di sistema e sviluppatori coinvolti nella gestione e nell'evoluzione del sistema.

1.2 Campo di applicazione

Il sistema **SeiPDF Management** è utilizzato per la gestione automatizzata del ciclo di vita dei documenti PDF generati da sistemi informativi interni.

Le funzionalità principali includono:

- estrazione e generazione dei documenti PDF a partire da database Oracle;
- aggregazione dei documenti in pacchetti compressi (ZIP) secondo vincoli dimensionali;
- preparazione dei file per l'invio verso sistemi esterni;
- gestione degli esiti di trasmissione.

Il sistema opera in ambiente server Windows ed è integrato con:

- servizi web REST sviluppati in **ASP.NET Core**;
- script batch schedulati tramite **PowerShell**;
- un servizio di invio file sviluppato in **Java** ed eseguito su **Apache Tomcat**;
- un database **Oracle** per la gestione dei dati applicativi.

1.3 Definizioni e acronimi

Termine	Descrizione
PDF	Portable Document Format
ZIP	Formato di compressione utilizzato per l'aggregazione dei documenti
API	Application Programming Interface
IIS	Internet Information Services
FTP	File Transfer Protocol
DB	Database

Scheduler	Processo schedulato per esecuzione automatica
Batch	Elaborazione automatica non interattiva

1.4 Riferimenti normativi o documentali

Specifica_Test_SeiPDF.pdf

2. Panoramica del sistema

2.1 Descrizione generale

Il sistema **SeiPDF Management** è una piattaforma applicativa dedicata alla gestione automatizzata del ciclo di vita dei documenti PDF, dalla loro estrazione fino alla trasmissione verso sistemi esterni e alla gestione degli esiti.

Il sistema è progettato per operare in modalità batch, attraverso processi schedulati che eseguono le seguenti attività:

- estrazione dei documenti PDF da database Oracle;
- scrittura dei file su filesystem;
- aggregazione dei documenti in archivi compressi (ZIP);
- invio dei pacchetti verso un sistema FTP esterno (Postel);
- acquisizione e gestione degli esiti tramite posta elettronica.

L'architettura è distribuita su più componenti applicativi, integrati tra loro tramite filesystem e database.

2.2 Contesto applicativo

Il sistema si inserisce in un contesto di integrazione con sistemi esterni, in particolare:

- **Database Oracle**, che rappresenta la fonte primaria dei dati e dei documenti (PDF in formato BLOB);
- **Server FTP Postel**, utilizzato per la trasmissione dei pacchetti documentali;
- **Casella di posta elettronica**, utilizzata per la ricezione degli esiti di invio.

Il sistema opera su infrastruttura **Windows Server 2022** ed è progettato per funzionare in modalità completamente automatizzata, senza intervento manuale.

2.3 Sistemi esterni coinvolti

Il sistema **SeiPDF Management** interagisce con diversi sistemi esterni necessari al completamento del ciclo documentale.

In particolare:

- **Database Oracle**
  Rappresenta la fonte primaria dei dati applicativi e dei documenti, contenuti in formato BLOB.
  Il sistema accede al database per l'estrazione dei PDF (e dei rispettivi metadati) e per l'aggiornamento degli stati dei lotti.
- **Server FTP Postel**
  Sistema esterno utilizzato per la trasmissione dei pacchetti documentali (file ZIP e file di

controllo `.t`).
Il trasferimento avviene tramite protocollo FTP mediante un servizio dedicato.

- **Sistema di posta elettronica**
  Utilizzato per la ricezione degli esiti di invio da parte del sistema Postel.
  Gli esiti vengono elaborati dal sistema per aggiornare lo stato dei documenti.

2.4 Stakeholder e attori

Il sistema coinvolge diversi attori, sia tecnici che operativi.

Attori tecnici

- **Amministratori di sistema**
  Responsabili della gestione dell'infrastruttura, della configurazione dei servizi (IIS, Tomcat, scheduler) e dei permessi di accesso.
- **Sviluppatori / manutentori applicativi**
  Responsabili dell'evoluzione del sistema, della manutenzione del codice e della gestione degli aggiornamenti.

Attori applicativi

- **Sistemi informativi interni**
  Generano i dati e i documenti PDF memorizzati nel database Oracle.
- **Sistema Postel**
  Riceve i pacchetti documentali e restituisce gli esiti di elaborazione.

Utenti operativi (indiretti)

Il sistema opera prevalentemente in modalità automatizzata; pertanto, non sono previsti utenti finali diretti.

Eventuali accessi manuali sono limitati a:

- consultazione e test tramite interfaccia Swagger;
- attività di monitoraggio e diagnostica.

3. Architettura del sistema

3.1 Architettura logica

Il sistema **SeiPDF Management** è strutturato secondo un'architettura a componenti, in cui ciascun modulo è responsabile di una specifica fase del ciclo documentale. I principali componenti logici sono:

- **Modulo di estrazione PDF**
  - Recupera i documenti dal database Oracle (BLOB)
  - Li trasforma in file PDF su filesystem
- **Modulo di creazione ZIP**
  - Aggrega i PDF secondo regole applicative
  - Applica vincoli dimensionali e numerici
  - Genera file .zip e file di controllo `.t`
- **Modulo di invio FTP**
  - Trasmette i pacchetti verso il sistema Postel
  - Gestisce eventuali errori di trasmissione
- **Modulo di gestione esiti**
  - Legge le comunicazioni ricevute via email
  - Aggiorna lo stato dei lotti nel database
- **Modulo di orchestrazione batch**
  - Coordina l'esecuzione dei processi tramite scheduler

L'interazione tra i componenti avviene tramite:

- **filesystem condiviso** (per lo scambio file)
- **database Oracle** (per lo stato applicativo)

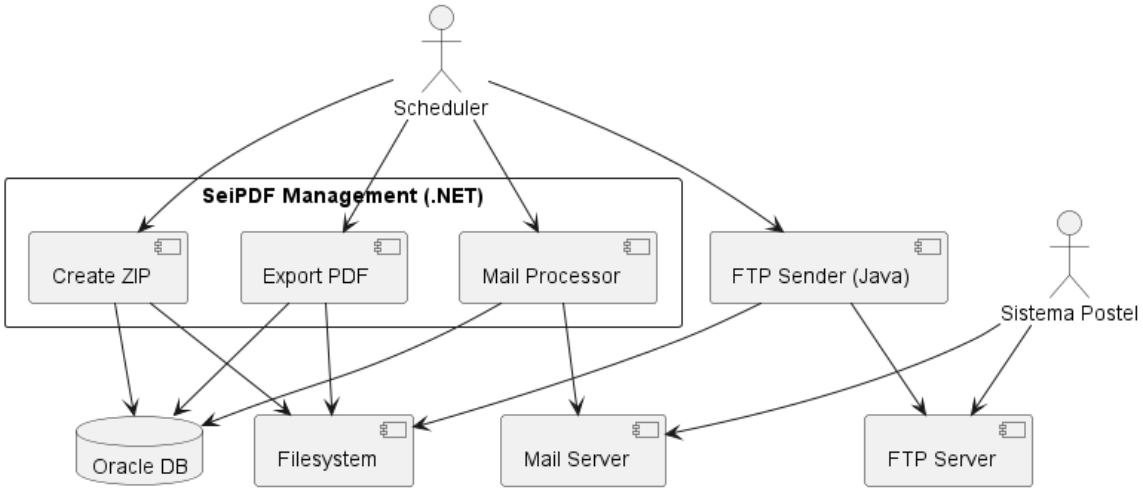


Immagine 1 – Architettura logica

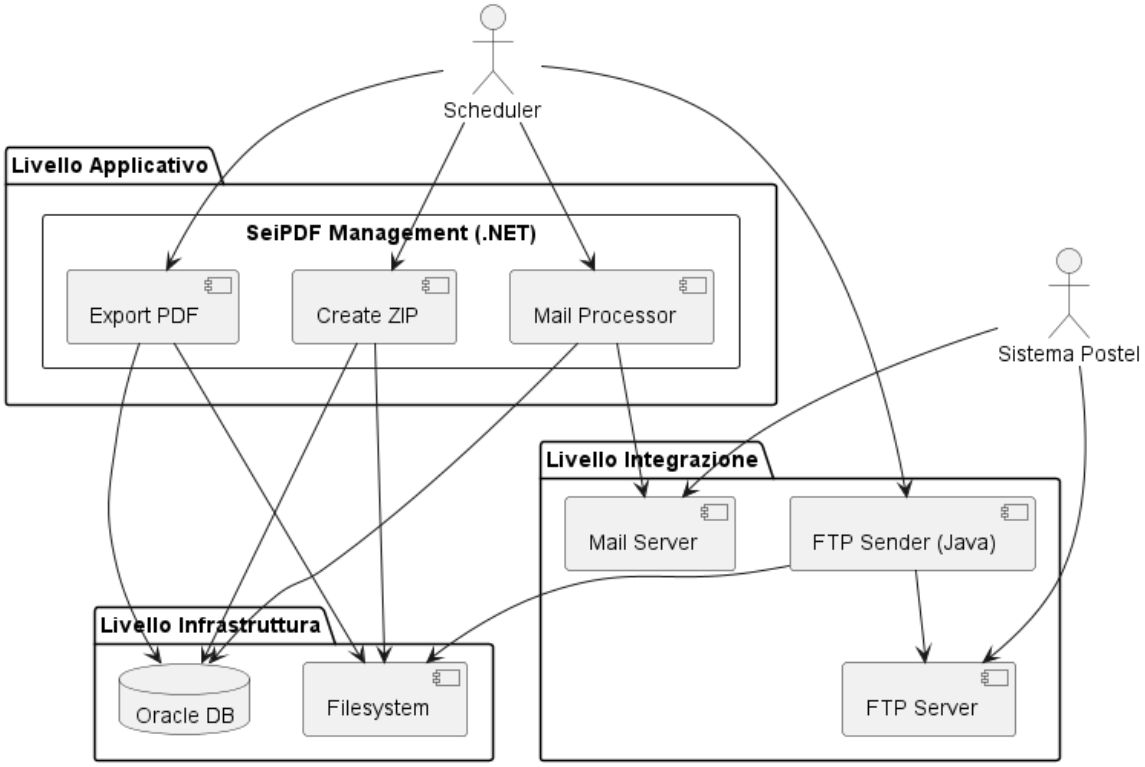


Immagine 2 – Architettura a livelli

Descrizione dell'architettura a livelli

Il sistema è organizzato secondo una suddivisione a livelli, che consente una chiara separazione delle responsabilità:

- **Livello applicativo**
  Contiene la logica di business implementata nell'applicazione SeiPDF Management, responsabile dell'elaborazione dei documenti e della gestione degli stati.
- **Livello di integrazione**
  Gestisce la comunicazione con sistemi esterni, inclusi:
  - il servizio di invio FTP
  - il server FTP Postel
  - il sistema di posta elettronica
- **Livello infrastrutturale**
  Comprende le risorse di base:
  - database Oracle per la persistenza dei dati
  - filesystem per lo scambio dei file tra i componenti

Questa separazione consente una maggiore modularità, facilitando manutenzione, evoluzione e isolamento dei problemi.

3.2 Architettura fisica

Il sistema è installato su infrastruttura **Windows Server 2022** ed è composto da più componenti distribuiti sullo stesso server applicativo.

L'applicazione principale è sviluppata in ASP.NET Core .NET 8 ed è ospitata su Microsoft IIS. I servizi complementari comprendono un componente Java per la trasmissione FTP, eseguito tramite Apache Tomcat, e processi batch gestiti attraverso Windows Task Scheduler.

Struttura filesystem

La struttura principale del sistema è la seguente:

```
E:\SEIPDF
│
├── App
│   └── SeiPdfManagement (publish .NET)
├── Backup
│   └── SeiPdfManagement
│       └── Backup dei rilasci applicativi
│
├── Logs
│   └── Log applicativi e scheduler
│
└── Scripts
    └── Script PowerShell e batch
```

Le directory svolgono le seguenti funzioni:

- **App**: contiene il deploy dell'applicazione .NET pubblicata da Visual Studio
- **Backup**: contiene gli archive ZIP delle versioni applicative precedent, generate prima delle operazioni di deployment
- **Logs**: contiene i log applicativi e dei processi batch
- **Scripts**: contiene gli script PowerShell e bash utilizzati per l'esecuzione schedulata dei processi

Il servizio di invio FTP:

- è deployato separatamente su **Apache Tomcat**
- viene distribuito come file **.war**
- risiede nella directory webapps di Tomcat

Hosting applicativo

- **SeiPDF Management**
  - ospitato su **IIS**
  - runtime **.NET 8**
  - modello **in-process, tramite ASP.NET Core Module V2**
- **Servizio FTP**
  - ospitato su **Apache Tomcat 9**
  - runtime **Java 21**

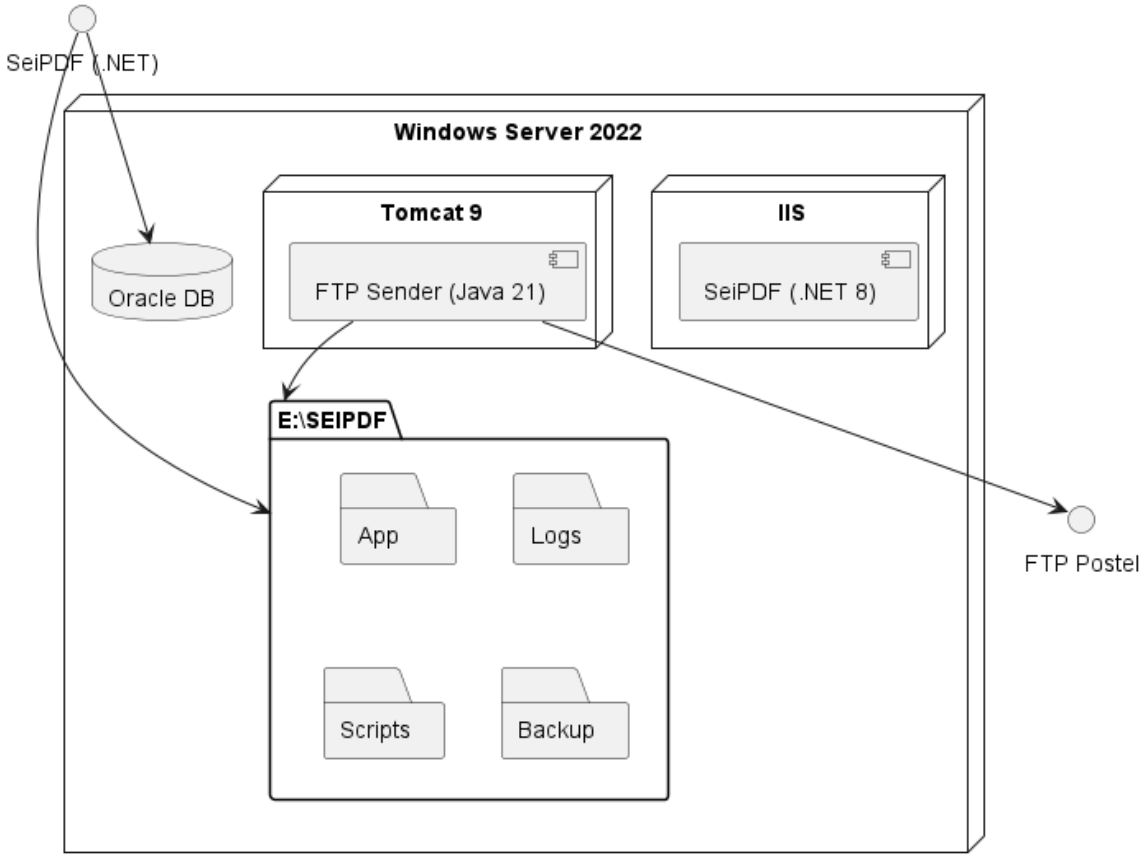


Immagine 3 – Architettura fisica

3.3 Componenti applicativi

Il sistema è composto dai seguenti componenti applicativi principali:

SeiPDF Management (.NET 8)

Applicazione backend che espone API REST per:

- estrazione PDF (`export-pdf`)
- creazione ZIP (`create-zip`)
- gestione esiti (`mail`)

Include:

- logging applicativo (Serilog)
- accesso database Oracle
- interfaccia Swagger (protetta da Windows Authentication + tabella utenti)

Scheduler (Windows Task Scheduler)

Gestisce l'esecuzione automatica dei processi tramite:

- script PowerShell:
  - export PDF
  - creazione ZIP
  - gestione mail
- script batch:
  - invio FTP

Servizio FTP (Java + Tomcat)

Applicazione Java che:

- legge i file ZIP e .t dal filesystem
- li invia al server FTP Postel
- gestisce eventuali errori di trasmissione

Database Oracle

Contiene:

- documenti PDF (BLOB)
- stato dei lotti
- log applicativi

Filesystem

Area condivisa tra i componenti per:

- scrittura PDF
- creazione ZIP
- gestione file `.t`

3.4 Tecnologie utilizzate

Componente	Tecnologia	Ruolo
Backend API	.NET 8 (ASP.NET Core)	Gestione logica applicativa e API REST
Web Server	IIS	Hosting applicazione .NET
Application Server Java	Apache Tomcat 9	Esecuzione servizio FTP
Linguaggio FTP	Java 21	Implementazione logica di invio file
Database	Oracle	Persistenza dati e stati
Scheduler	Windows Task Scheduler	Esecuzione automatica dei job
Script	PowerShell / Batch	Automazione invocazione processi
Logging	Serilog	Gestione logging applicativo
Protocollo invio	FTP	Trasferimento file verso Postel

4. Componenti software

4.1 API SeiPDF (ASP.NET Core)

Il componente principale del sistema è l'applicazione **SeiPDF Management**, sviluppata in **.NET 8 (ASP.NET Core)** e ospitata su **IIS**.

L'applicazione espone una serie di endpoint REST utilizzati esclusivamente dai processi batch per l'esecuzione delle operazioni applicative.

Responsabilità principali

- estrazione dei documenti PDF dal database Oracle;
- creazione dei pacchetti ZIP secondo regole configurate;
- gestione degli esiti ricevuti via email;
- aggiornamento dello stato dei lotti nel database;
- logging delle operazioni applicative.

Endpoint principali

- `POST /api/seipdf/export-pdf`
  Estrazione dei PDF dal database e scrittura su filesystem
- `POST /api/seipdf/create-zip`
  Creazione archivi ZIP e file `.t`
- `POST api/mail/download-unread`
  Lettura email e aggiornamento stati

Caratteristiche tecniche

- hosting su **IIS** con modello **in-process**;
- utilizzo di **Serilog** per il logging su file;
- accesso a database Oracle tramite driver dedicato;
- configurazione tramite file `appsettings.json`;
- presenza di interfaccia **Swagger** per test e documentazione API.

Sicurezza

- accesso a Swagger protetto tramite **Windows Authentication** + **tabella utenti**;
- API operative destinate esclusivamente a esecuzione automatizzata.

4.2 Scheduler PowerShell

L'esecuzione dei processi applicativi è gestita tramite **Windows Task Scheduler**, che invoca gli endpoint REST tramite script PowerShell.

Responsabilità principali

- esecuzione automatica e periodica degli endpoint;
- gestione dei timeout delle chiamate API;
- logging delle esecuzioni batch;
- gestione degli errori di esecuzione.

Script principali

- **Export PDF**
  - invoca endpoint `export-pdf`
  - frequenza elevata (es. ogni 5 minuti)
- **Creazione ZIP**
  - invoca endpoint `create-zip`
  - aggregazione documenti
- **Gestione Mail**
  - invoca endpoint `mail`
  - aggiornamento stati

Caratteristiche tecniche

- utilizzo di `Invoke-RestMethod` per chiamate HTTP;
- gestione header (eventuale API Key);
- logging su file dedicati;
- configurazione centralizzata dei percorsi e URL.

4.3 Servizio FTP Java su Tomcat

Il servizio di invio FTP è un'applicazione sviluppata in **Java 21**, deployata su **Apache Tomcat 9** sotto forma di file `.war`.

Responsabilità principali

- lettura dei file ZIP e .t dal filesystem;
- invio dei file al server FTP Postel;
- gestione degli errori di trasmissione;
- logging delle operazioni di invio.

Modalità di esecuzione

- invocato tramite script `.bat` schedulato;
- eseguito come applicazione web su Tomcat;
- interazione diretta con filesystem locale.

Caratteristiche tecniche

- runtime **Java 21**;
- deploy su **Tomcat 9** (directory `webapps`);
- utilizzo protocollo **FTP** per trasferimento file;
- gestione retry e logging lato applicativo.

4.4 Database Oracle

Il database Oracle rappresenta il sistema di persistenza centrale per il sistema SeiPDF.

Responsabilità principali

- memorizzazione dei documenti PDF in formato BLOB;
- gestione dello stato dei lotti documentali;
- tracciamento delle operazioni applicative;
- supporto alla gestione degli esiti.

Tipologie di dati gestiti

- documenti PDF (BLOB);
- identificativi dei lotti ZIP;
- associazione ZIP → PDF;
- stati di elaborazione e invio;
- log applicativi.

Interazione con il sistema

- accesso in lettura durante l'estrazione PDF;
- accesso in scrittura durante:
    - creazione ZIP (associazione documenti)
    - gestione esiti (aggiornamento stati)

4.5 Filesystem di integrazione

Il filesystem locale rappresenta il punto di integrazione tra i diversi componenti del sistema.

Particolare attenzione è posta alla gestione dei permessi sul filesystem, al fine di garantire la corretta interoperabilità tra i processi IIS e il servizio Java in esecuzione su Tomcat.

Responsabilità principali

- memorizzazione temporanea dei PDF estratti dal database;
- creazione e gestione dei file ZIP;
- generazione dei file di controllo `.t`;

- scambio file tra applicazione .NET e servizio FTP Java;
- gestione dello stato dei file tramite spostamento tra directory.

Struttura delle directory

Le directory operative si trovano sullo stesso livello della cartella principale del sistema:

```
E:\
│
├── SEIPDF
│    ├── App
│    ├── Logs
│    └── Scripts
│
├── DaInviare
└── Inviati
```

Directory operative

DaInviare

Contiene:

- PDF generati dall'endpoint `export-pdf`;
- file ZIP creati dall'endpoint `create-zip`;
- file di controllo `.t` associati agli ZIP.

Questa directory rappresenta **l'area di lavoro attiva del sistema.**

Inviati

Contiene:

- file ZIP già trasmessi al server FTP;
- file .t associati.

Questa directory rappresenta **l'archivio dei file già processati.**

Flusso dei file

Il ciclo di vita dei file sul filesystem è il seguente:

1. I PDF vengono estratti dal database e salvati in `DaInviare`;
2. I PDF vengono aggregati in file ZIP (sempre in `DaInviare`);
3. Per ogni ZIP viene creato il relativo file `.t`;
4. Il servizio FTP legge ZIP e `.t` da `DaInviare`;
5. Dopo l'invio:
    - i file vengono spostati nella directory `Inviati`.

Caratteristiche operative

- accesso concorrente da:
    - applicazione .NET (IIS)
    - servizio Java (Tomcat)
- gestione dei permessi:
    - necessaria configurazione condivisa tra IIS AppPool e processo Java
    - accesso in lettura/scrittura su directory operative
- gestione dei file:
    - cancellazione PDF dopo zippatura
    - spostamento ZIP dopo invio FTP

5. Flussi applicativi

5.1 Generazione PDF

Il processo di generazione PDF ha lo scopo di estrarre i documenti dal database Oracle e renderli disponibili sul filesystem per le successive fasi di elaborazione.

Flusso operativo

1. Il sistema legge i record dalla vista applicativa (`VIEW_H2H_SEI_PDF`);
2. Per ogni record:
    - viene recuperato il contenuto del documento (BLOB);
    - vengono estratti i metadati necessari al naming;
3. Il file PDF viene salvato nella directory `DaInviare`.

Regole di naming

Il nome del file viene costruito secondo il formato:

```
FRONTERETRO_CENTRODICOSTO + ELABORAZIONE_TIPO_SERVIZIO.pdf
```

Esempio:

```
FR_C2_812590_CONV_COLON_SP.pdf
```

derivato direttamente dai campi della view

Gestione stati database

Per ogni documento:

- successo:
    - `STATO = 50`
    - `SEIPDF = 1`
    - `INVIATO = 1`
- errore (es. BLOB nullo):
    - `STATO = 92`

Logging

- inserimento log tecnico su:
  - `H2H_LOG_SEIPDF`
  - `H2H_SEIPDF_LOG`

5.2 Creazione pacchetti ZIP

Il processo di creazione ZIP aggrega i documenti PDF presenti nella directory operativa in archivi compressi, secondo regole applicative.

Flusso operativo

1. Il sistema legge i PDF presenti in `DaInviare`;
2. I file vengono raggruppati secondo pattern:

```
^(FR_|FF_)(C\d+)_
```

quindi per:

- tipo documento (FR / FF)
- centro di costo

Regole di creazione ZIP

- nome ZIP:

```
PM_<GRUPPO>_<SEQUENCE>.zip
```

Esempio:

```
PM_FR_C2_0002757.zip
```

Vincoli applicati

- dimensione massima ZIP (configurabile)
- numero massimo file per ZIP
- stima compressione:

```
dimensione stimata = PDF * 0.25 + overhead
```

per evitare ZIP troppo piccoli o troppo grandi

Operazioni per ogni ZIP

- creazione archivio ZIP
- aggiunta file PDF
- creazione file .t
- aggiornamento DB:

```
ZIP → PDF contenuti
```

- eliminazione PDF dal filesystem

Gestione stati

- inserimento log:

```
"File generato"
```

5.3 Invio file tramite FTP

Il processo di invio FTP ha lo scopo di trasferire i pacchetti documentali verso il sistema esterno Postel.

Il servizio è implementato tramite un'applicazione Java deployata su Apache Tomcat ed è invocato tramite script batch schedulato.

Flusso operativo

1. Il servizio accede alla directory `DaInviare` e individua:
   - file `.zip`
   - file `.t`
2. I file `.zip` vengono:
   - ordinati per nome
   - inviati uno alla volta al server FTP
3. L'invio avviene tramite comando esterno `curl` con protocollo FTPS:

```
curl -v --ssl-reqd -k -T <file> ftp://user:pass@server:port/path
```

4. Per ogni file:
   - viene eseguito il comando di upload;
   - viene acquisito l'esito (exit code del processo);
5. In caso di invio riuscito:

- viene registrato un log su database;
- viene aggiornato lo stato del lotto (INVIATO = 1);
- viene inserito un log tecnico di tracciamento;
- il file viene spostato nella directory Inviati;

lo spostamento è effettuato in modo robusto con:

- retry multipli
- attesa di rilascio file
- gestione attributi read-only

Gestione file `.t`

I file `.t` vengono gestiti separatamente rispetto ai file `.zip`.

Per ogni file `.t`:

1. viene verificata la presenza del corrispondente file `.zip` nella directory `Inviati`;
2. il file `.t` viene inviato **solo se lo ZIP associato è già stato trasferito con successo**;

questo comportamento garantisce:

- coerenza dei dati inviati
- rispetto del protocollo di integrazione con Postel

Gestione errori

In caso di errore:

- il file NON viene spostato;
- rimane nella directory `DaInviare`;
- verrà ritentato automaticamente al ciclo successivo;

Logging e tracciamento

Il sistema registra:

- log su file (per debugging operativo);
- log su database:
  - stato invio
  - log tecnico

- messaggi di errore

Output

- file trasferiti al server FTP Postel;
- file archiviati in:

`E:\Inviati`

5.4 Gestione esiti

Il processo di gestione esiti consente l'aggiornamento dello stato dei lotti in base alle risposte ricevute dal sistema Postel.

Attivazione

- Endpoint: `mail`
- Modalità: script PowerShell schedulato

Flusso operativo

1. Il sistema si connette al server IMAP (Office365) tramite OAuth2;
2. recupera le email non lette;
3. filtra le email in base all'oggetto (parole chiave configurate);

Elaborazione email

Per ogni email:

- se l'oggetto è rilevante:
  - vengono estratti gli allegati;
  - gli allegati vengono salvati su filesystem;
  - il contenuto viene letto (XML);
  - viene processato tramite componente applicativo;
- l'email viene marcata come letta;

Aggiornamento sistema

- parsing XML esito;
- aggiornamento stato lotti su database;
- inserimento log applicativi.

Caratteristiche tecniche

- autenticazione OAuth2 (client credentials)
- connessione IMAP TLS
- gestione limite massimo email per run
- logging su DB e file

5.5 Diagramma di sequenza

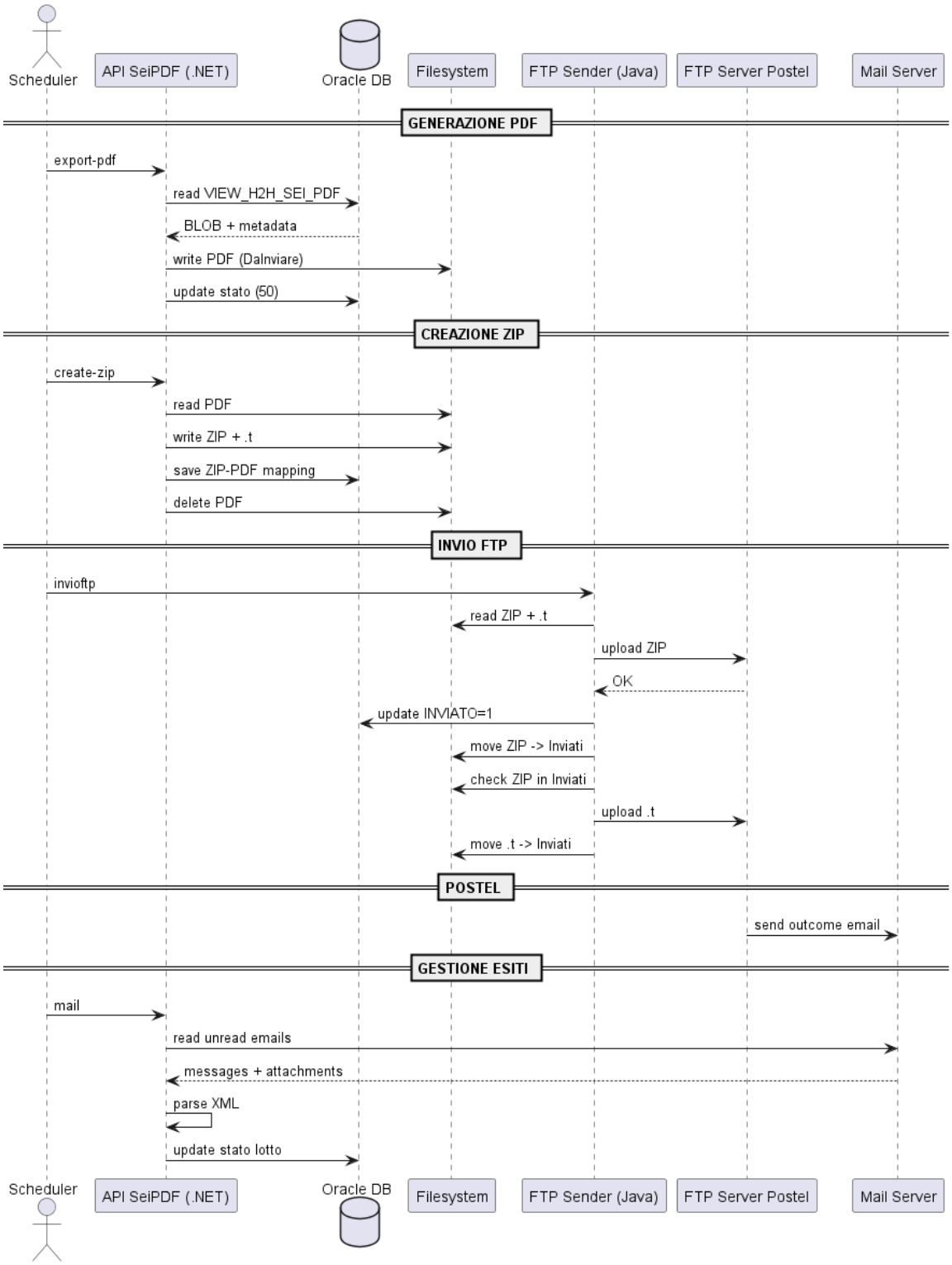


Immagine 4 – Diagramma di sequenza

5.6 Diagramma a stati

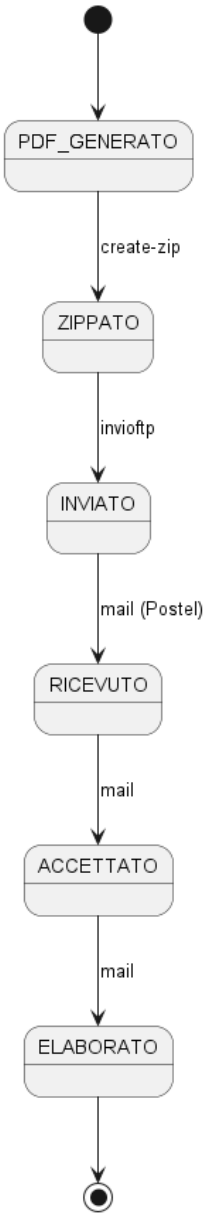


Immagine 5 – Stati di elaborazione

Descrizione degli stati

Il ciclo di vita di un lotto documentale è il seguente:

- **PDF_GENERATO**
  Il documento è stato estratto dal database e salvato su filesystem;
- **ZIPPATO**
  Il documento è stato incluso in un archivio ZIP;
- **INVIATO**
  Il file ZIP è stato trasmesso al sistema Postel;
- **RICEVUTO**
  Il sistema Postel ha ricevuto il lotto;
- **ACCETTATO**
  Il lotto è stato validato dal sistema destinatario;
- **ELABORATO**
  Il lotto è stato completamente processato.

Eventuali anomalie non generano stati applicativi, ma vengono gestite tramite comunicazioni esterne.

5.7 Diagramma di flusso

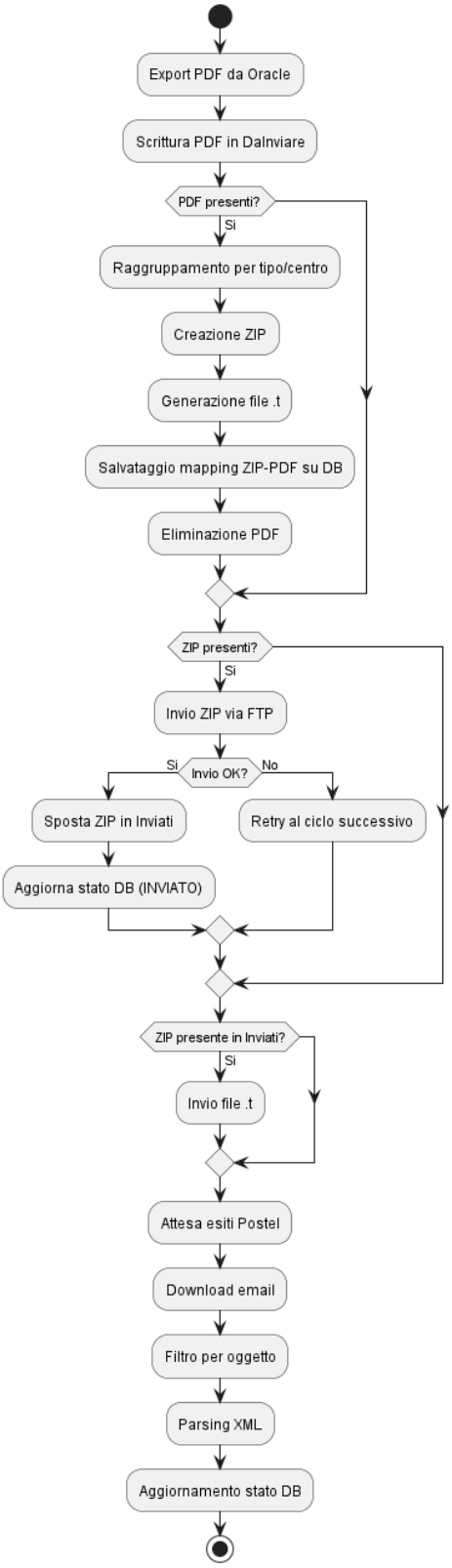


Immagine 6 – digramma di flusso

Descrizione

Il diagramma rappresenta il flusso operativo completo del sistema SeiPDF, dalla generazione dei documenti fino alla gestione degli esiti.

Il processo è strutturato in fasi sequenziali ma indipendenti, consentendo la riesecuzione automatica in caso di errori o interruzioni.

6. API REST

6.1 Elenco endpoint

L'applicazione SeiPDF Management espone i seguenti endpoint REST:

Metodo	Endpoint	Descrizione	Autenticazione
GET	/health	Verifica stato applicazione (DB e filesystem)	Nessuna
GET	/api/Mail/download-unread	Scarica ed elabora le email non lette	API Key
POST	/api/seipdf/export-pdf	Estrazione dei PDF dal database Oracle	API Key
POST	/api/seipdf/create-zip	Creazione archivi ZIP e file .t	API Key

---

6.2 Parametri di input

Body

Tutti gli endpoint **NON prevedono body di input**

Header opzionale

Per alcuni endpoint (es. mail):

```
X-API-KEY: <valore>
```

autenticazione opzionale

Configurazione runtime

Gli endpoint utilizzano configurazioni interne:

- `appsettings.json`
- configurazioni DB
- directory filesystem
- parametri scheduler

6.3 Struttura delle risposte

Le API restituiscono JSON strutturati con **riepilogo dell'esecuzione batch**.

Health

```
{
  "status": "Healthy",
  "application": "SeiPDF Management",
  "timestampUtc": "2026-04-28T10:00:00Z",
  "oracle": "OK",
  "filesystem": {
    "DaInviare": "OK",
    "Inviati": "OK"
  },
  "errors": []
}
```

stato completo sistema

Export PDF

```
{
  "total": 100,
  "exported": 95,
  "nullBlob": 3,
  "errors": 2,
  "files": [
    "E:\\DaInviare\\file1.pdf"
  ]
}
```

Descrizione:

- `total` → record letti
- `exported` → PDF creati
- `nullBlob` → BLOB nulli
- `errors` → errori runtime
- `files` → file generati

Create ZIP

```
{
  "totalPdfFound": 150,
  "zipCreated": 5,
  "pdfZipped": 120,
  "zipFiles": [
    "E:\\DaInviare\\PM_FR_C2_00001.zip"
```

```
  ]
}
```

significato:

- `totalPdfFound` → PDF trovati
- `zipCreated` → ZIP creati
- `pdfZipped` → PDF inseriti
- `zipFiles` → file prodotti

Mail (download-unread)

```
{
  "success": true,
  "count": 10,
  "files": [
    "E:\\Mail\\file1.xml"
  ]
}
```

significato:

- `success` → esito operazione
- `count` → numero file elaborati
- `files` → file salvati

6.4 Gestione errori

Il sistema utilizza una gestione errori standard basata su HTTP e struttura RFC.

Codici HTTP

Codice	Significato
200	Operazione completata
401	API Key non valida
500	Errore interno
503	Servizio non disponibile (health)

Formato errore

```
{
  "type": "https://tools.ietf.org/html/rfc9110",
  "title": "An error occurred while processing your request.",
  "status": 500,
  "detail": "Descrizione errore",
```

```
  "traceId": "..."
}
```

Gestione resilienza

- gli errori non interrompono l'intero batch;
- i record in errore vengono conteggiati;
- i file non processati vengono ripresi nei cicli successivi;
- logging completo su:
  - file
  - database

6.5 Endpoint applicativi non esposti tramite Swagger

Il sistema include endpoint HTTP utilizzati esclusivamente per l'esecuzione batch e non documentati tramite interfaccia Swagger.

Invio FTP

Metodo	Endpoint	Descrizione
POST	/inviaFtp	Invio file ZIP e .t verso server FTPS Postel

Descrizione

Il servizio è implementato come **servlet Java** deployata su Apache Tomcat.

L'endpoint è invocato tramite script batch schedulato e ha il compito di:

- inviare file ZIP e `.t` al server FTP Postel;
- gestire l'ordine di invio;
- garantire la coerenza tra ZIP e file `.t`;
- spostare i file elaborati nella directory `Inviati`.

Modalità di utilizzo

L'endpoint può essere invocato in due modalità:

---

1. Invio massivo

Se non viene specificato alcun parametro:

- vengono inviati tutti i file presenti nella directory `DaInviare`;
- i file `.zip` vengono inviati per primi;
- i file `.t` vengono inviati solo se lo ZIP associato è già presente in `Inviati`.

---

2. Invio singolo file

È possibile specificare il file da inviare tramite JSON:

```
{
  "nomeFile": "PM_FR_C2_0002757.zip"
}
```

Regole operative

- i file `.zip` sono ordinati alfabeticamente prima dell'invio;
- i file `.t` sono inviati solo dopo conferma dello ZIP associato;
- i file vengono spostati nella directory `Inviati` dopo invio riuscito;
- i file non inviati rimangono in `DaInviare`.

Output

L'endpoint restituisce una risposta testuale contenente:

- elenco file processati;
- eventuali skip (es. .t senza ZIP associato);
- messaggi di errore.

Logging

Il servizio utilizza:

- logging su file (configurato dinamicamente per data);
- logging dettagliato per ogni operazione di invio.

7. Scheduler e processi batch

7.1 Descrizione job schedulati

Il sistema SeiPDF Management utilizza il **Windows Task Scheduler** per l'esecuzione automatica dei processi batch.

I job schedulati orchestrano l'intero ciclo documentale invocando:

- endpoint REST dell'applicazione .NET;
- servizi applicativi esterni (invio FTP su Tomcat).

Elenco job

Job	Descrizione	Frequenza
Export PDF	Estrazione dei documenti dal database Oracle e scrittura su filesystem	Ogni 5 minuti
Create ZIP	Aggregazione dei PDF in archivi compressi e generazione file .t	Ogni 30 minuti
Invio FTP	Trasferimento dei file ZIP e .t al server Postel	Ogni 30 minuti
Gestione Mail	Lettura delle email e aggiornamento degli stati dei lotti	Ogni 60 minuti

---

Sequenza logica

I job seguono una pipeline:

```
Export PDF → Create ZIP → Invio FTP → Gestione Mail
```

ogni fase è indipendente ma concatenata logicamente tramite filesystem e database.

7.2 Script PowerShell

Gli endpoint dell'applicazione .NET vengono invocati tramite script PowerShell.

Caratteristiche principali

- utilizzo di `Invoke-RestMethod` per chiamate HTTP;
- gestione timeout elevati (es. fino a 30 minuti);
- gestione header (eventuale API Key);
- logging su file dedicati;

- gestione errori tramite blocchi `try/catch`.

Script principali

Export PDF

- endpoint: `/api/seipdf/export-pdf`
- funzione:
  - invoca il servizio di estrazione PDF;
  - registra il numero di file generati.

Create ZIP

- endpoint: `/api/seipdf/create-zip`
- funzione:
  - invoca la creazione dei pacchetti ZIP;
  - registra i file prodotti.

Gestione Mail

- endpoint: `/api/Mail/download-unread`
- funzione:
  - scarica ed elabora le email;
  - aggiorna lo stato dei lotti.

Invio FTP

Il processo di invio FTP è gestito tramite:

- script `.bat`;
- invocazione della servlet Java su Tomcat (`/inviaFtp`).

7.3 Frequenza esecuzione

I job sono configurati con le seguenti frequenze:

Job	Frequenza	Tipo esecuzione
Export PDF	Ogni 5 minuti	PowerShell (Task Scheduler)
Create ZIP	Ogni 30 minuti	PowerShell (Task Scheduler)
Invio FTP	Ogni 30 minuti	Batch + Servlet Java (Tomcat)
Gestione Mail	Ogni 60 minuti	PowerShell (Task Scheduler)

Considerazioni operative

- la frequenza elevata dell'export garantisce aggiornamento continuo dei dati;
- i processi successivi operano su batch accumulati;
- il sistema è progettato per tollerare esecuzioni ripetute (idempotenza).

7.4 Logging scheduler

Ogni processo schedulato produce log dedicati per consentire monitoraggio e troubleshooting.

Tipologie di log

- log script PowerShell:
  - registrazione avvio/fine processo;
  - esito chiamate API;
  - eventuali errori;
- log applicativi:
  - prodotti dall'applicazione .NET (Serilog);
  - prodotti dal servizio Java FTP;
- log database:
  - stato dei lotti;
  - eventi applicativi.

Struttura log

Esempio:

```
2026-04-28 10:00:00 [INFO] START Export PDF
2026-04-28 10:00:10 [INFO] PDF esportati: 95
2026-04-28 10:00:10 [INFO] END Export PDF
```

Obiettivi del logging

- tracciabilità completa dei processi;
- supporto al debugging;
- verifica corretto funzionamento del sistema;
- audit delle operazioni effettuate

8. Configurazione sistema

8.1 Configurazione IIS

L'applicazione **SeiPDF Management** è ospitata su **IIS (Internet Information Services)** in ambiente Windows Server 2022.

Modalità di hosting

L'applicazione è sviluppata mediante ASP.NET Core .NET 8 e utilizza il modello di hosting **in-process**.

In tale configurazione, il codice applicativo viene eseguito direttamente all'interno del processo worker di IIS (w3wp.exe), tramite ASP.NET Core Module V2.

Il modello in-process consente di evitare il passaggio attraverso un processo Kestrel separato, semplificando l'architettura di hosting.

Configurazione sito

Parametro	Valore
Nome sito IIS	SeiPdfManagement
Server	PVESBASL5
Protocollo	HTTP
Porta	8089
Directory fisica	`E:\SEIPDF\App\SeiPdfManagement`
Application Pool	`SeidPdf`
Endpoint health	`/health`

Application Pool

L'applicazione utilizza un Application Pool dedicato, con le seguenti caratteristiche:

- Nome: `SeidPdf`.
- .NET CLR Version: No Managed Code.
- Managed Pipeline Mode: Integrated.
- Hosting ASP.NET Core: in-process.

L'identità dell'Application Pool deve disporre dei permessi necessari per l'accesso alle risorse applicative.

In particolare, sono richiesti permessi adeguati sulle directory operative:

- `E:\DaInviare`
- `E:\Inviati`
- `C:\temp\seipdfmanagement_logs`
- `E:\email-attachments`

Configurazione applicative e variabili d'ambiente

Le configurazioni dell'applicazione sono gestite mediante il file `appsettings.json` e le variabili d'ambiente associate all'Application Pool IIS.

I parametri sensibili, tra cui credenziali di accesso al database Oracle, parametri di autenticazione Microsoft 365 e API Key, sono definiti tramite variabili d'ambiente, evitando la memorizzazione di valori riservati nei file versionati.

ASP.NET Core utilizza la convenzione con doppio carattere di sottolineatura (__) per rappresentare la struttura gerarchica delle impostazioni.

Ad esempio:

```
Security__ApiKey
```

Le variabili d'ambiente dell'Application Pool rimangono indipendenti dai file applicativi distribuiti dalla pipeline CI/CD.

Autenticazione

- API operative:
  - accesso anonimo abilitato;
- Swagger:
  - protetto tramite **Windows Authentication**;
  - eventuale controllo utenti su database.

Logging

- logging applicativo tramite Serilog;
- directory configurata:

```
C:\temp\seipdfmanagement_logs
```

configurazione da appsetting.json

Aggiornamento applicativo

Il rilascio delle nuove versioni è effettuato tramite Gitea Actions.

Durante il deployment viene utilizzato il file `app_offline.htm` per consentire la temporanea sospensione dell'applicazione senza richiedere l'arresto manuale dell'Application Pool.

Terminata la copia, il file viene rimosso e la pipeline verifica la disponibilità del servizio tramite l'endpoint:

```
http://pvesbasl5:8089/health
```

In caso di anomalie, è previsto un tentativo automatico di ripristino della versione precedente.

8.2 Configurazione Tomcat

Il servizio di invio FTP è deployato su **Apache Tomcat 9** come applicazione Java.

Deploy applicazione

- file: `.war`
- directory:

```
<TOMCAT_HOME>\webapps\
```

- endpoint esposto:

```
http://<server>:<porta>/invioFtp
```

Configurazione FTP

I parametri di connessione sono definiti nel file ftp-config.properties

---

Parametri principali

```
ftp.server=server_postel
ftp.port=2110
ftp.remoteDir=SEIPDFINSERVICE
```

Directory locali

```
local.folder=E:\\DaInviare\\
sent.folder=E:\\Inviati\\
```

Strumenti di invio

Il servizio utilizza:

```
curl.path=C:\\tools\\curl-openssl\\bin\\curl.exe
```

per eseguire upload FTPS

Logging

```
log.folder=C:\\temp\\
```

- log giornalieri generati automaticamente;
- nome file:

```
ftp-upload-YYYY-MM-DD.log
```

8.3 Configurazione Oracle

Il sistema utilizza un database **Oracle** come sistema di persistenza centrale.

Connessione

Configurata tramite stringa di connessione in appsetting.json

```
HOST=host_db
PORT=port_db
SERVICE_NAME=SIA2
USER=SEIPDF
```

Utilizzo

Il database è utilizzato per:

- lettura documenti PDF (BLOB);
- aggiornamento stati;
- gestione lotti;
- logging applicativo.

Accessi richiesti

L'utente applicativo deve avere:

- SELECT su viste applicative;
- INSERT/UPDATE su tabelle di stato;
- accesso alle procedure di supporto.

8.4 Parametri applicativi

I parametri applicativi sono definiti nel file appsetting.json

---

Sicurezza

```
"Security": {
  "ApiKey": "SEIPDF-INTERNAL-2026"
}
```

- utilizzata per autenticazione opzionale API.

Export PDF

```
"SeiPdfExport": {
  "OutputDirectory": "E:\\DaInviare",
  "MaxRecords": 0
}
```

- directory output PDF;
- limite record (0 = nessun limite).

Creazione ZIP

```
"SeiPdfZipSettings": {
  "InputDirectory": "E:\\DaInviare",
  "MaxZipSizeMb": 15,
  "MaxFilesPerZip": 250,
  "CreateMultipleZipsPerRun": true
}
```

- dimensione massima ZIP;
- numero massimo file;
- gestione multi-zip.

Mail

```
"MailSettings": {
  "ImapServer": "outlook.office365.com",
  "ImapPort": 993
}
```

Autenticazione

```
TenantId, ClientId, ClientSecret
```

OAuth2 (Microsoft)

Filtri email

```
"SubjectsOfInterest": [...]
```

- utilizzati per identificare email rilevanti.

Directory allegati

```
"AttachmentDir": "e:\\email-attachments"
```

Logging

```
"LogDirectory": "C:\\temp\\seipdfmanagement_logs"
```

Tutti i parametri applicativi sono configurabili senza necessità di modifica del codice, consentendo adattabilità a diversi ambient

9. Sicurezza

9.1 Gestione accessi

L'accesso al sistema SeiPDF Management è differenziato in base alla tipologia di componente.

- **API REST**
  - accessibili in modalità anonima per consentire l'esecuzione batch;
  - utilizzate esclusivamente da processi schedulati interni;
- **Interfaccia Swagger**
  - protetta tramite **Windows Authentication**;
  - accesso consentito solo a utenti autorizzati;
- **Servizio FTP**
  - autenticazione tramite credenziali configurate;
  - connessione sicura tramite protocollo **FTPS**;
- **Accesso database**
  - tramite utente dedicato (`SEIPDF`);
  - credenziali gestite tramite configurazione applicativa.

9.2 Permessi filesystem

Il filesystem rappresenta un punto critico di integrazione tra componenti.

Sono richiesti permessi di:

- lettura/scrittura su:
  - `E:\DaInviare`
  - `E:\Inviati`
  - directory log

I processi coinvolti sono:

- IIS (Application Pool)
- servizio Java (Tomcat)

è necessario garantire accesso concorrente senza conflitti.

Particolare attenzione deve essere posta alla configurazione dei permessi, al fine di evitare problemi di accesso ai file durante le operazioni di scrittura, lettura e cancellazione.

9.3 Protezione API

Le API sono progettate per uso interno e non esposte pubblicamente.

Misure adottate:

- accesso limitato a rete interna;
- eventuale utilizzo di API Key (`X-API-KEY`);
- logging completo delle operazioni;
- assenza di input utente esterni.

10. Performance e limiti

10.1 Limiti dimensionali ZIP

Il sistema applica vincoli per garantire compatibilità con il sistema Postel:

- dimensione massima ZIP: 15 MB

- dimensione calcolata tramite:
  - dimensione reale filesystem;
  - stima compressione dei PDF.

10.2 Numero massimo documenti

Per ogni archivio ZIP:

```
MaxFilesPerZip = 250
```

configurabile via `appsettings.json`.

10.3 Timeout processi

I processi batch sono configurati per gestire operazioni lunghe:

- timeout chiamate API:
  - fino a **30 minuti** (PowerShell);
- invio FTP:
  - dipendente da dimensione file e rete;
- gestione mail:
  - limitata da numero massimo email per esecuzione.

Caratteristiche di resilienza

- processi idempotenti;
- retry automatico tramite scheduler;
- gestione indipendente dei singoli record.

11. Deployment

11.1 Installazione API su IIS

Il deployment dell'applicazione **SeiPDF Management**, sviluppata in ASP.NET Core .NET 8, è gestito mediante una pipeline CI/CD implementata attraverso **Gitea Actions** ed eseguita su un runner Windows.

La procedura automatizza la compilazione, la verifica delle dipendenze, la generazione degli artefatti e la distribuzione dell'applicazione sul server IIS, prevedendo meccanismi di backup, verifica dell'integrità e rollback in caso di anomalie.

Ambiente di destinazione

Parametro	Configurazione
Server applicativo	PVESBASL5 – Windows Server 2022
Web server	Microsoft IIS
Framework	ASP.NET Core .NET 8
Sito IIS	SeiPdfManagement
Application Pool	SeidPdf
Modalità Application Pool	No Managed Code
Hosting model	In-process
Directory applicativa	`E:\SEIPDF\App\SeiPdfManagement`
Endpoint di verifica	`http://pvesbasl5:8089/health`
Repository	Gitea – SeiPDFManagement
Branch di produzione	`V1.1`

Pipeline CI/CD

La pipeline è definita nel repository Git attraverso il file:

`.gitea/workflows/build.yml`

Il workflow viene attivato dagli eventi configurati nel repository, mentre la fase di deployment è autorizzata esclusivamente per i push sul branch di produzione `V1.1`, in presenza di modifiche che richiedano l'aggiornamento applicativo.

Fasi della pipeline

1. Recupero del codice sorgente

Il runner Windows effettua il checkout del repository Gitea, recuperando la revisione del codice associata all'esecuzione.

2. Compilazione e pubblicazione

L'applicazione viene compilata in configurazione Release mediante .NET CLI, attraverso le operazioni:

- `dotnet restore`
- `dotnet build -c Release`
- `dotnet publish -c Release`

I file pubblicabili vengono generati in una directory temporanea del runner.

3. Generazione SBOM

Viene prodotto un Software Bill of Materials (SBOM) nel formato CycloneDX 1.5, contenente l'inventario delle dipendenze NuGet dell'applicazione.

4. Analisi delle vulnerabilità

Le dipendenze software vengono analizzate mediante OWASP Dependency-Check.

La scansione utilizza un database locale delle vulnerabilità e prevede una soglia di controllo CVSS pari a 7.

I risultati sono prodotti nei formati HTML e JSON.

5. Archiviazione degli artefatti

La pipeline archivia sul runner:

- il pacchetto ZIP contenente i file pubblicabili;
- il documento SBOM;
- i report dell'analisi delle vulnerabilità.

Gli artefatti sono organizzati per branch, data di esecuzione e identificativo del commit.

6. Backup preventivo

Prima di aggiornare l'applicazione, viene eseguita una copia della directory di produzione.

Il contenuto viene archiviato in formato ZIP nella directory:

```
E:\SEIPDF\Backup\SeiPdfManagement
```

Per il backup viene calcolato un hash SHA-256, utilizzato come riferimento per la verifica dell'integrità dell'archivio.

7. Pubblicazione automatizzata su IIS

Il runner stabilisce una connessione SMB autenticata con il server applicativo.

La distribuzione viene effettuata nella directory:

```
E:\SEIPDF\App\SeiPdfManagement
```

Prima della sincronizzazione viene creato il file `app_offline.htm`, che consente ad ASP.NET Core Module di mettere temporaneamente offline l'applicazione.

La copia dei file avviene tramite Robocopy, utilizzando la modalità di sincronizzazione `/MIR`.

Al termine della pubblicazione viene verificata la corrispondenza dell'hash SHA-256 della DLL principale tra staging e destinazione.

Il file `app_offline.htm` viene quindi rimosso, consentendo la riattivazione dell'applicazione.

8. Verifica dello stato applicativo

Dopo la pubblicazione viene invocato l'endpoint:

```
GET http://pvesbasl5:8089/health
```

Il controllo verifica la disponibilità dell'applicazione e lo stato dei componenti essenziali, tra cui:

- connessione al database Oracle;
- accessibilità delle directory operative;
- servizio di esportazione PDF;
- servizio di creazione ZIP;
- servizio di gestione email;
- assenza di errori segnalati dall'endpoint.

Sono previsti tentativi ripetuti per consentire il completamento dell'avvio applicativo.

9. Rollback automatico

Qualora il deployment o le verifiche successive non vadano a buon fine, la pipeline tenta il ripristino della versione precedente utilizzando lo snapshot creato prima della pubblicazione.

La procedura comprende:

- messa offline dell'applicazione;
- ripristino dei file precedenti;
- riattivazione dell'applicazione;
- nuova verifica dell'endpoint di health check.

In caso di mancato ripristino, l'errore viene registrato nei log della pipeline e viene richiesto un intervento tecnico.

Gestione delle configurazioni sensibili

Le credenziali e i parametri sensibili necessari all'applicazione sono configurati tramite variabili d'ambiente associate all'Application Pool IIS.

Le credenziali utilizzate dalla pipeline per accedere al server applicativo sono gestite mediante i secret del repository Gitea.

Tracciabilità dei rilasci

Ogni esecuzione della pipeline consente di risalire al commit distribuito, ai risultati dei controlli di sicurezza, agli artefatti generati e all'esito del deployment.

Questa modalità garantisce maggiore ripetibilità delle operazioni di rilascio, riduzione degli interventi manuali e disponibilità di una procedura di ripristino in caso di anomalie.

11.2 Installazione servizio Java su Tomcat

- build file `.war`;
- deploy in:

```
<TOMCAT_HOME>\webapps\
```

- avvio automatico tramite Tomcat.

11.3 Configurazione scheduler

- utilizzo di Windows Task Scheduler;
- configurazione job:
  - export PDF
  - create ZIP
  - invio FTP
  - gestione mail;
- associazione script:
  - PowerShell
  - `.bat`

11.4 Configurazione connessione database

- configurazione in `appsettings.json`;
- parametri:
    - host
    - porta
    - service name
    - credenziali.

12. Requisiti di sistema

Requisiti hardware

- CPU: minimo 2 core (consigliato 4+)
- RAM: minimo 8 GB (consigliato 16 GB)
- Disco:
    - spazio sufficiente per gestione PDF/ZIP
    - I/O veloce (SSD consigliato)

Requisiti software

- Windows Server 2022
- IIS
- .NET 8 Runtime
- Java 21
- Apache Tomcat 9
- Oracle Client

13. Ambiente di esercizio

Il sistema **SeiPDF Management** è installato in ambiente di produzione su infrastruttura aziendale, composta da un server applicativo principale e da sistemi esterni integrati.

13.1 Componenti dell'ambiente

L'ambiente di esercizio è costituito dai seguenti sistemi:

Server applicativo

- **Sistema operativo**: Windows Server 2022 (pvesbasl5)
- **Funzione**:
  - esecuzione applicazione SeiPDF Management (.NET 8 su IIS)
  - esecuzione servizio FTP Java (Tomcat)
  - gestione script batch e scheduler
  - gestione filesystem applicativo

Server database

- **Tecnologia**: Oracle Database (SIA2)
- **Funzione**:
  - memorizzazione documenti PDF (BLOB)
  - gestione stati dei lotti
  - logging applicativo

Server FTP Postel

- **Nome**: rmftp1.postel.com
- **Tecnologia**: FTPS (FTP su TLS)
- **Funzione**:
  - ricezione file ZIP e .t
  - gestione del processo di recapito

Sistema di posta elettronica

- **Tecnologia**: Microsoft 365 (IMAP)
- **Server**: outlook.office365.com
- **Funzione**:
    - ricezione esiti da Postel
    - gestione aggiornamenti stato lotti

Filesystem applicativo

- **Tipologia**: storage locale su server applicativo (pvesbasl5)
- **Funzione**:
    - repository temporaneo dei PDF
    - gestione file ZIP e .t
    - area di integrazione tra componenti

13.2 Modalità di esecuzione

Il sistema opera in modalità completamente automatizzata tramite:

- **Windows Task Scheduler**
    - esecuzione periodica dei processi batch;
- **Script PowerShell**
    - invocazione endpoint API (.NET);
- **Script batch (.bat)**
    - invocazione servizio FTP Java;

Schedulazione

- esecuzione continua durante la giornata;
- frequenze differenziate per ciascun processo;
- riesecuzione automatica in caso di errore.

13.3 Considerazioni operative

- il sistema è progettato per operare **senza interfaccia utente diretta**;
- tutte le operazioni sono eseguite in modalità batch;
- il filesystem funge da meccanismo di integrazione tra componenti;
- i log applicativi consentono il monitoraggio delle esecuzioni;

- eventuali anomalie sono gestite tramite:
    - retry automatici;
    - logging dettagliato;
- il sistema è idempotente e tollera esecuzioni ripetute.

13.4 Sintesi componenti

Componente	Tecnologia	Posizione	Ruolo
API SeiPDF	.NET 8 / IIS	Server applicativo	Gestione logica applicativa e API REST
FTP Sender	Java 21 / Tomcat	Server applicativo	Invio file verso server Postel
Database	Oracle	Server DB	Persistenza dati e stati
Mail	Office365	Esterno	Invio esiti e notifiche
FTP	Postel	Esterno	Ricezione file ZIP e .t